FAST — NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES
FAST School of Computing · Karachi Campus · Department of Artificial Intelligence & Data Science
AL-2002 Artificial Intelligence (Laboratory) · Fall 2026

PROJECT PROPOSAL

Semester Lab Project · Weightage 1 mark

Group Information

	Name	Roll Number	Section	Role in the project
Leader	Faiza Usmani	24k-2006	BCY-5A	Leader — AI algorithm and evaluation
Member 2	Rabia Hakeem	24k-2042	BCY-5A	Board/game logic and testing
Member 3	Wajeeha Habibullah	24k-2004	BCY-5A	GUI and documentation

Project Title	Connect Four AI
Project Idea	Idea No. _____ from the Project Ideas list ☐ Own idea (needs approval) ☑
Category	Part A — Game ☑ Part B — Applied / Data ☐
GitHub Repository URL	https://github.com/Faiza-usmani/Connect-Four-
Date of Submission	21 October 2026

1. Project Title and Summary

Connect Four AI is a graphical game in which users can play either against another person or against a computer-controlled opponent. The game uses a standard 7-column by 6-row board, and players take turns dropping pieces into a column. In Human vs. AI mode, the computer uses Minimax with Alpha-Beta Pruning to examine possible moves and choose one that improves its chances of winning while responding to the opponent's threats. A heuristic evaluation function estimates the strength of board positions that are not yet won or lost. Players can select different AI search depths, and the game will display wins, losses, and draws. The project demonstrates how adversarial search enables a program to make strategic decisions instead of choosing moves randomly.

2. Problem Statement and Motivation

Connect Four appears simple because each turn consists of choosing a column, but a strong move depends on both the current board and the opponent's possible replies. The board has 42 cells, and each column can contain pieces from either player in a constrained order. Even though the board is small, the number of possible move sequences grows rapidly as the game progresses; searching every possible continuation is therefore impractical beyond shallow depths. A move that creates an immediate opportunity may also allow the opponent to complete a horizontal, vertical, or diagonal line of four. A useful game-playing agent must recognize immediate wins, block immediate losses, and assess less obvious patterns such as two-way threats and central control. Human players can reason about these possibilities, while a basic computer opponent may rely on random legal moves and miss important threats. This project addresses that gap by implementing adversarial search and a heuristic that estimates the strategic value of non-terminal positions. The result is worthwhile because it provides a playable game and a practical demonstration of how AI search improves decision-making in a competitive environment.

3. Problem Formulation

3.1 PEAS Description

Component	Description for your project
Performance measure	Win rate against the baseline and human players; number of wins, losses, and draws; legal-move correctness; nodes explored and decision time.
Environment	A standard 7-column × 6-row Connect Four board with two alternating players. The environment is fully visible and follows fixed rules.
Actuators	Drop a piece into a selected column; update the board; show game status; restart the game; select mode and AI difficulty.
Sensors	Current board cells, legal columns, current player, winning lines, full-board/draw condition, and selected game mode/difficulty.

3.2 Environment and Agent Type

Property	Classification — tick one
Observability	Fully observable ☑ Partially observable ☐
Determinism	Deterministic ☑ Stochastic ☐
Change over time	Static ☑ Dynamic ☐
State and actions	Discrete ☑ Continuous ☐
Agents	Single-agent ☐ Multi-agent / adversarial ☑
Agent type	Simple reflex ☐ Model-based ☐ Goal-based ☑ Utility-based ☑ Learning ☐

3.3 Problem Representation

Element	Your representation
State representation	A 6 × 7 grid where each cell is empty, Human/Player 1, or AI/Player 2, plus the current player and game status.
Actions / moves	Choose any column that is not full and drop the current player's piece into its lowest empty cell.
Initial state	An empty 6 × 7 board, with Player 1 taking the first turn.
Goal or terminal test	A player wins by forming four pieces in a horizontal, vertical, or diagonal line; the game is a draw if the board is full without a win.
Cost, utility, fitness or evaluation	Terminal win/loss/draw states receive very high positive/negative/zero scores. Non-terminal states are scored using centre-column preference, counts of two/three pieces in four-cell windows, and penalties for opponent threats. Earlier wins are preferred.

4. Proposed AI Approach

#	Technique	Role in the project	Own code or library
1	Minimax	Searches future turns assuming both players choose moves to maximize their own outcome; selects the best move at a fixed depth.	Own Python implementation
2	Alpha-Beta Pruning	Skips branches that cannot change the Minimax decision, allowing deeper search with fewer node evaluations.	Own Python implementation
3	Heuristic evaluation + move ordering	Scores non-terminal boards and considers promising/central moves earlier to improve practical search efficiency.	Own Python implementation

#	Technique	Role in the project	Own code or library
B	Baseline:	Chooses uniformly from currently legal columns; used as a simple comparison baseline.	Own Python implementation

Connect Four is a deterministic, turn-based, adversarial game with a discrete state space, so Minimax is a suitable method for selecting moves by considering both the AI's choices and the opponent's responses. Alpha-Beta Pruning preserves Minimax's result while avoiding branches that cannot affect the final decision, making a depth-limited search more efficient. Since the full game tree is too large to explore completely, the agent will stop at a configurable depth and evaluate the resulting non-terminal boards. The heuristic will reward centre-column occupancy and windows containing two or three AI pieces with open spaces, while penalizing equivalent opponent opportunities. Terminal wins and losses will receive much larger scores than ordinary board patterns, and the score will prefer faster wins and delayed losses. A random legal-move agent will provide a baseline for measuring the improvement.

4.1 Planned Class Design

• Board: stores the 6 × 7 grid and current player; methods include get_valid_moves(), drop_piece(), check_winner(), is_draw(), and copy().
• Game: manages turns, game mode, end conditions, and restart; methods include play_move(), switch_turn(), and reset_game().
• Agent (base class): defines the choose_move(board) interface; RandomAgent and MinimaxAgent will implement it.
• RandomAgent: selects a legal column at random; used as the baseline opponent.
• MinimaxAgent: stores search_depth and evaluation settings; methods include choose_move(), minimax(), alpha_beta(), and evaluate_board().
• ConnectFourGUI: renders the board and controls; handles clicks, mode/difficulty selection, status messages, and displaying the AI's chosen move.
Relationship: Game owns the Board and calls an Agent when the AI must move; ConnectFourGUI sends user actions to Game and redraws the resulting state.

5. Scope

In scope — the features you commit to delivering. List the features that must work for the project to be considered complete.

#	Feature	Why it is essential
1	Playable 7 × 6 board	Implements the standard rules and legal piece dropping.
2	Human vs. AI mode	Allows a person to play against the Minimax agent.
3	Human vs. Human mode	Allows two people to play on the same device.
4	Minimax, Alpha-Beta Pruning, and heuristic scoring	Demonstrates the project's core AI methods.
5	Win/draw detection, restart, and difficulty selection	Makes the game usable and enables repeatable comparisons at different search depths.

Out of scope — deliberately excluded. Naming what you will not build is as important as naming what you will; it shows that your scope is realistic for a group of two.

Online multiplayer, accounts, networking, and matchmaking; machine-learning training or neural networks; a perfect solver that searches the entire game tree; advanced animations or sound; and saving player profiles or long-term statistics. These features are not required to demonstrate adversarial search and would increase implementation time.

6. Data or Environment Source

Source name and URL	Generated / simulated Connect Four board states; no external dataset or URL is required.
Type	Public dataset ☐ Web API ☐ Self-collected ☐ Generated / simulated ☑
Access requirements	No key needed ☑ Free key required ☐ (Paid sources are not permitted)

Approximate size	One standard 6 × 7 board per game. For evaluation, use at least 100 seeded AI-vs-random games per configuration, plus selected hand-designed tactical positions.
Licence / terms of use	The board states and game engine will be generated by the team; no third-party dataset licence is required. Standard game rules are implemented directly.
Offline fallback	The application and all generated test cases will be stored locally in the repository. Gameplay and experiments will run without a network connection.

> ***Required:*** your program must still run when the network is unavailable, by reading a locally cached copy of any data it downloads. Describe your caching plan in the last row above.

7. Evaluation Plan

What will be compared	Minimax + Alpha-Beta Pruning at selected search depths versus a random legal-move baseline; optionally compare two Minimax depths.
Metric(s)	Win/draw/loss rates, average nodes evaluated, and average AI decision time.
Baseline	RandomAgent selecting uniformly from legal columns.
Test instances / test set	Same starting empty board and same game rules; seeded repeated AI-vs-random matches, with AI and baseline taking turns first in balanced batches. Include tactical positions for immediate win/block checks.
Number of runs and seed	At least 100 games per configuration, using fixed documented seeds and identical conditions; report results across the same batch.
Planned charts (at least three)	1) Win/draw/loss rate by AI depth. 2) Average nodes evaluated by search depth. 3) Average decision time by search depth.

8. Division of Work

Member	Modules / components owned	Approximate share
Faiza Usmani (24k-2006)	Minimax and Alpha-Beta Pruning implementation; heuristic evaluation; AI testing and performance analysis.	34%
Rabia Hakeem (24k-2042)	Board/game logic; legal moves and win/draw detection; integration tests and rule validation.	33%
Wajiha Habibullah (24k-2004)	GUI, game-mode and difficulty controls; status display; documentation, plots, and final integration support.	33%

9. Milestones and Timeline

#	Milestone — what will be demonstrably working	Target date	Owner
1	Board engine supports legal moves, turn switching, win/draw detection, and restart.	Week 1	Rabia
2	Human vs. Human mode and basic GUI are playable from start to terminal state.	Week 2	Wajiha + Rabia
3	Minimax, heuristic evaluation, and Alpha-Beta Pruning select legal AI moves at configurable depths.	Week 3	Faiza
4	Random baseline and seeded experiment runner report wins, nodes, and decision time.	Week 4	Faiza + Rabia
5	Experiments complete, final integration, documentation and repository cleanup	Week 5	All members

10. Expected Outcome and Risks

The completed submission will provide a graphical Connect Four game with Human vs. Human and Human vs. AI modes, a restart option, win/draw messages, and selectable AI search depth. The AI will use Minimax with Alpha-Beta Pruning and a heuristic that values useful four-cell patterns, centre control, and blocking opponent threats. Evaluation will compare it with a random legal-move agent using the same game conditions and fixed seeds, reporting win/draw/loss rates, nodes evaluated, and decision time. Two main risks are excessive search time at higher depths and bugs in win detection or board updates. If search is slow, the team will reduce the default depth and improve move ordering; if rule bugs occur, it will add targeted unit tests for horizontal, vertical, and diagonal cases.

For office use — Lab Instructor	
Proposal status	Approved ☐ Approved with changes ☐ Revision required ☐
Comments	
Signature and date	

End of Project Proposal